Bucket-level dye and Hodge projection clause selection: a +17 lift over `mzr02` on chainy MPTP, and what the HJB–NS refactor revealed

Mettapedia Working Notes

June 4, 2026

Abstract

We report a clause-selection heuristic, *ABGC* (Advective / Bidirectional / Geodesic Control), which projects each E prover unprocessed clause onto a 14-bit bucket signature, maintains a bidirectional dye field $(a_{\text{fwd}}, a_{\text{bwd}})$ over buckets together with a directed flow u_{ij} across parent→child bucket transitions, and applies a discrete Hodge projection at batch boundaries before reading off a per-clause weight. Interleaved at the 20 % slot of a 12-multiplier round-robin alongside AI4REASON's `mzr02` Mizar-tuned strategy in E's HCB scheduler, the controller solves 289/800 on the validation slice of extended MPTP 5k (`chainy_val_clean`) under a 30 s CPU limit per problem, four parallel jobs, and a per-process 10 GiB virtual-memory cap. The matched `mzr02`-alone single-heuristic baseline solves 272/800 on the same slice; the combined heuristic therefore lifts the baseline by +17 problems (+6.2 % relative). E's portfolio `--auto-schedule`, included for context only, solves 334/800.

A subsequent principled refactor mapped the controller onto the Hamilton–Jacobi–Bellman / Navier–Stokes (HJB–NS) skeleton of Goertzel and Stay: conjugate-gradient Hodge projection with isolated-node handling, mass-conserving advection with a per-node CFL share, Laplacian viscosity on the net-flow field, and graph-Laplacian diffusion of the dyes with a degree-aware stability cap. The refactor was implemented end-to-end, its conservation laws verified by telemetry, and a SEGV-class crash was diagnosed and fixed (per-node CFL violation → negative dye → log(neg) → NaN in the clause heap → corrupted comparator → SIGSEGV in `ClauseSetExtractEntry`). The HJB–NS refactor did *not* reproduce the +17 lift: the best Sub-phase A configuration (conjugate-gradient projection without transport) achieves

253/800, and fully-wired Sub-phase B transport achieves 246–238/800, depending on the projection sign convention.

We characterize the cause: the static-H Phase 4 hybrid implicitly reads a *divergence-amplified* signal in u_{ij} that the clean Helmholtz projection removes by design. The amplification was carried by the under-converged Jacobi solver plus a sign convention on the projection correction that adds rather than subtracts the gradient component. The controller's H-term shape (`meet`, `align`, `pressure`) was implicitly tuned to that signal regime, and the principled transport produces a structurally different signal that the current H-term does not exploit. We give this finding the status it deserves: clean math beats accidental signal only when the consumer is also rebuilt to read the cleaner signal.

Contents

1 Introduction

E prover's saturation loop is parametrised by a heuristic that assigns each unprocessed clause a real weight; a min-heap selects the next clause to process. The choice of heuristic shapes which proofs are found within a fixed time budget. AI4REASON's `mzr02` strategy (Jakubův and Urban, IJCAR 2020) is a Mizar-tuned hand-engineered heuristic which on our benchmark slice (`chainy_val_clean`, 800 problems from the extended MPTP 5k validation set) solves 272/800 in 30 s per problem. This is our single-heuristic baseline.

The Heuristic Control Block (HCB) mechanism in E permits round-robin *interleaving* of multiple weight functions inside the same prover process: every clause is scored by every heuristic, and the scheduler rotates through them with configurable multipliers. Crucially, all heuristics share the same unprocessed-clause set, so a selection made by heuristic A generates child clauses that heuristic B may later select. In this paper, "+17 over `mzr02`" will always mean: an interleaved configuration which mixes `mzr02`'s 9 entries (multiplier sum 9) with one ABGC entry at multiplier 3, producing a 9:3 rotation (25 % ABGC clause picks). The score is a property of the *interleaved* process, not of either heuristic in isolation. Solo ABGC scores far below either baseline (Sec. 3.5).

We address two questions:

- **Does ABGC, as interleaved with `mzr02`, beat `mzr02`-alone?** Yes, by +17 problems (+6.2 % relative).
- **Does a principled Hamilton–Jacobi–Bellman / Navier–Stokes reformulation of ABGC further improve the result?** No, in this

paper's empirical regime: the principled transport loses the lift. We characterize the cause as a signal/consumer mismatch.

Both findings are reproducible from the artifacts listed in Sec. 8.

Sibling paper. This paper is the saturation-loop sibling of our premise-selection paper *PLN as a Common Substrate for Naive Bayes and k-NN Premise Selection* (`papers/PLN-kNN-NB.tex`), which addresses the upstream question of *which premises* to feed E on the same MPTP 5k slice. The two scopes do not overlap: PLN-kNN-NB chooses the axioms; ABGC chooses, within saturation, which clauses to process next.

Layout. Sec. 2 describes the ABGC controller as a 14-bit bucket abstraction with a static H-term and a Hodge-projected flow. Sec. 3 reports the headline empirical result: 289/800 in the hybrid, with set-diff analysis and a solo-ABGC ablation. Sec. 4 reports the HJB–NS principled refactor: eight Sub-phase A configurations sweeping the solver, sign convention, source budget, and pressure signal; three Sub-phase B configurations wiring viscosity, advection, and diffusion; and the SEGV diagnosis. Sec. 5 characterises why the cleaner controller underperforms the accident: the consumer (the H-term) was implicitly tuned to a signal regime that the principled controller removes by construction. Sec. 6 situates the work alongside MaLARea / MaSh / ENIGMA and the watchlist tradition. Sec. 7 states what is open. Sec. 8 gives the heuristic string, the runner invocation, the CSV paths backing each reported number, and the build hash.

2 The ABGC architecture

ABGC is implemented as a single E weight function, `AdvWeight`, in `HEURISTICS/che_abgc.c` (the source file referenced throughout, with header `HEURISTICS/che_abgc.h`). The controller has four parts: a bucket abstraction over clauses, a per-bucket dye field, a directed parent-to-child flow on the bucket graph, and an H-term that combines static clause features with three flow-derived signals.

2.1 Bucket signatures

Every clause is projected onto a 14-bit signature; this gives an address space of $2^{14} = 16384$ logical buckets, but the controller allocates at most `ABGC_BMAX` $= 1024$ active bucket slots, hashed from the 14-bit signature with an

overflow slot at (`ABGC_BMAX` -1). The signature packs the following features into bit fields of width $\{1, 1, 1, 3, 3, 3, 1, 1\}$:

- is-goal (1 bit) — did this clause come from a negated conjecture path?
- is-unit (1 bit) — is the clause a single literal?
- is-ground (1 bit) — are all terms ground?
- depth bin (3 bits) — 8 bins of clause proof-depth
- size bin (3 bits) — 8 bins of clause weight (log-spaced)
- conjecture-overlap bin (3 bits) — 8 bins of symbol-overlap with the conjecture's relevant function-symbol set
- polarity-shape lo / hi (2 bits) — coarse shape of the positive/negative literal distribution.

The hash from the 14-bit signature to a `ABGC_BMAX` slot uses Fowler–Noll–Vo (FNV-1a) modulo (`ABGC_BMAX` -1); collisions land in the overflow slot. This gives the controller a bounded state-vector (at most ~ 1024 active buckets) regardless of the number of clauses E touches per problem.

2.2 Per-bucket dye fields

Each active bucket b carries two non-negative scalars,

$$a_{\mathrm{fwd}}(b), \quad a_{\mathrm{bwd}}(b) \in [0, \infty),$$

called the forward and backward dyes. *Forward* dye is injected at bucket b when an inserted clause is typed as Axiom or Hypothesis; *backward* dye is injected when the clause is typed as Conjecture or NegConjecture. Both dyes decay multiplicatively at every batch boundary:

$$a_\bullet(b) \;\mapsto\; (1 - \gamma_a \, \Delta t)\, a_\bullet(b), \tag{1}$$

with $\gamma_a = 0.05$ and $\Delta t = 1.0$ by default. A bucket with both dyes appreciably positive is a *bridge bucket* — a place where axiom-rooted and goal-rooted derivations meet.

2.3 Bucket-to-bucket flow

For every clause inserted into the unprocessed set, the controller examines the clause's derivation; if the derivation has a recorded first parent, the parent clause's bucket p and the child clause's bucket b are linked by incrementing $u_{p,b}$, a sparse flow matrix of dimension `ABGC_BMAX` $\times$ `ABGC_BMAX`. The flow decays multiplicatively each batch with $\gamma_u = 0.08$.

The net flow is the antisymmetric part

$$F_{ij} = u_{ij} - u_{ji}. \tag{2}$$

At each batch boundary the controller applies a discrete Hodge projection to F on the active-bucket subgraph (Sec. 2.5). The projected F is redistributed into a corrected u which is then used by the H-term.

2.4 The H-term

Per-clause weight is the standard E ranking: lower weight wins. ABGC reads off a real $H(\text{clause})$ which the caller subtracts from a base weight. H is a static linear combination of seven features:

$$\begin{aligned} H \;=\; & w_{\text{conj}}\, h_{\text{conj}} + w_{\text{depth}}\, h_{\text{depth}} + w_{\text{size}}\, h_{\text{size}} + w_{\text{goal}}\, h_{\text{goal}} \\ & + w_{\text{meet}}\, h_{\text{meet}} + w_{\text{align}}\, h_{\text{align}} + w_{\text{pressure}}\, h_{\text{pressure}}, \end{aligned} \tag{3}$$

where

$$h_{\text{conj}} = \texttt{conjecture\_overlap\_count}(\text{clause}) \tag{4}$$

$$h_{\text{depth}} = -\,\texttt{proof\_depth}(\text{clause}) \tag{5}$$

$$h_{\text{size}} = -\,\log_2\bigl(1 + \texttt{weight}(\text{clause})\bigr) \tag{6}$$

$$h_{\text{goal}} = [\,\texttt{is\_goal}(\text{clause})\,] \tag{7}$$

$$h_{\text{meet}} = \log\bigl(\varepsilon + a_{\text{fwd}}(b) \cdot a_{\text{bwd}}(b)\bigr) \tag{8}$$

$$h_{\text{align}} = u_{p \to b} \quad (p = \texttt{parent\_bucket}(\text{clause})) \tag{9}$$

$$h_{\text{pressure}} = -\,p_{\text{Leray}}(b), \tag{10}$$

with the Leray pressure $p_{\text{Leray}}(b)$ produced as the dual variable of the Hodge projection (Sec. 2.5). The weights $(w_{\text{conj}}, w_{\text{depth}}, w_{\text{size}}, w_{\text{goal}}, w_{\text{meet}}, w_{\text{align}}, w_{\text{pressure}})$ are supplied through E's heuristic configuration string; the configuration used for all results in this paper is given in Sec. 8.

2.5 Discrete Hodge projection

We solve a Poisson equation on the active-bucket subgraph. Let $L = D - A$ be the graph Laplacian with $A_{ij} = 1$ if either u_{ij} or u_{ji} is non-zero and $D = \mathrm{diag}(\deg)$. For each batch, define

$$\mathrm{div}(F)[i] \;=\; \sum_{j>i:\, A_{ij}=1} F_{ij}. \tag{11}$$

We solve

$$L\,p \;=\; \mathrm{div}(F) \tag{12}$$

in the zero-mean subspace on the *active* sub-graph (vertices with $\deg \geq 1$); isolated vertices are excluded from the system and their p component is fixed at zero. The Phase 4 hybrid uses Jacobi iteration with a cap of 50 iterations; the HJB–NS refactor uses conjugate gradient on the same null-space-corrected system. The corrected flow is

$$F^{\mathrm{sol}}_{ij} \;=\; F_{ij} - (p_j - p_i), \quad u^{(\mathrm{new})}_{ij} \;=\; \tfrac{1}{2}(u_{ij}+u_{ji}) + \tfrac{1}{2}F^{\mathrm{sol}}_{ij}, \;\; u^{(\mathrm{new})}_{ji} \;=\; \tfrac{1}{2}(u_{ij}+u_{ji}) - \tfrac{1}{2}F^{\mathrm{sol}}_{ij}. \tag{13}$$

The sign convention $-(p_j - p_i)$ *adds* the divergent component of F rather than removing it; the Helmholtz-correct projection would use $+(p_j - p_i)$. This sign choice is mathematically wrong but empirically useful (Sec. 5); the alternative is examined in Sec. 4.

2.6 HCB interleaving with `mzr02`

Outside the AdvWeight function, the heuristic string passed to E is a 9:3 multiplier sum

```
(9mzr02entries)+3*AdvWeight(...)
```

which causes E's round-robin scheduler to pick from `mzr02`'s weight functions 9 out of every 12 selections and from `AdvWeight` 3 out of every 12. The unprocessed-clause set is shared across all entries. Bucket-state and Hodge-projected flow are recomputed lazily at every `batch_period` = 128 insertions, so the cost amortizes to roughly a single bucket-graph Hodge solve per ~ 100 clause-level operations.

3 Act 1 — empirical achievement

3.1 Dataset, prover, hardware

The benchmark slice is `chainy_val_clean` from the extended MPTP 5k dataset (originally derived from MML 5.94.1493 via the chainy filter; we

Configuration	Solved	Notes
E `--auto-schedule` (portfolio)	334 / 800	strategy ceiling, not a single-heuristic baseline
`mzr02` alone	272 / 800	single Mizar-tuned heuristic, our reference baseline

Table 1: Baselines on `chainy_val_clean`, 30 s CPU, 4 jobs. The portfolio number `--auto-schedule` is included so the reader does not mistake 272 as the prover's empirical ceiling; portfolio schedulers route across many heuristics and are not the comparator for a single-heuristic improvement claim.

Configuration	Solved	vs. `mzr02`-alone	vs. portfolio
`mzr02` alone	272 / 800	—	−62
`mzr02` + ABGC (9:3 HCB hybrid)	**289 / 800**	**+17**	−45
+ 6.2 % rel. over `mzr02`		+17	

Table 2: Single-heuristic baseline vs. the ABGC–`mzr02` hybrid on the same 800-problem slice. CSV: `atps/logs/abgc_mzr02_hybrid_chainy_val_30s.csv` (289); `atps/logs/baseline_mzr02_chainy_val_30s.csv` (272).

use the same 800-problem slice as the PLN-kNN-NB sibling paper for direct comparability). Per-problem CPU limit is 30 s, imposed both at the runner level (`timeout70s` hard wall) and inside E (`--cpu-limit=30`). All runs use four parallel workers (`parallel--jobs4`). Each worker has a 10 GiB virtual-memory cap (`ulimit-v10485760`). E's binary is built from `/home/zar/claude/eprover/` with the ABGC source files linked into the heuristics library; the relevant flags and build steps are recorded in Sec. 8.

3.2 Baselines

3.3 Hybrid result

The ABGC–`mzr02` hybrid solves 289/800, lifting the single-heuristic baseline by +17 problems (+6.2 % relative). Reproducibility is via the runner script invocation in Sec. 8; the CSVs cited in Table 2 exist on disk and a per-problem solved-count `awk` pass reproduces the headline number exactly.

3.4 Set-diff: where the +17 comes from

The +17 is a *net*: the hybrid solves some problems that `mzr02`-alone does not, and vice-versa. Per-problem set-diff on the two CSVs gives:

- hybrid-unique wins: 35 problems
- `mzr02`-unique wins: 18 problems
- shared wins: 254 problems.

Net: $35 - 18 = +17$.

The hybrid-unique wins are not concentrated in any narrow MPTP family; they spread across `compts`, `connsp`, `funct`, `relset`, and `zfmisc` prefixes. The `mzr02`-unique losses are similarly spread. The lift is a genuine breadth effect, not a narrow win on a particular kind of problem.

3.5 Solo-ABGC ablation

Removing `mzr02` from the heuristic string and running ABGC alone collapses the score. Solo-ABGC peaks in our exploration at roughly 200/800, well below either `mzr02`-alone (272) or the hybrid (289). This is not a failure of the controller; it is a feature of the design. ABGC's dye field is injected only at axiom and conjecture clauses, so derived clauses carry dye only through bucket-level transfer (parent→child flow), which is a sparse and noisy signal. In a hybrid, `mzr02` provides the dense exploration; ABGC's role is to bias toward bridges — buckets where forward and backward dyes meet — which mzr02 by itself does not target. The HCB rotation lets ABGC's "unusual" picks generate children that `mzr02` then selects three rounds later. The +17 is an interleaving effect, not a solo property.

4 Act 2 — HJB–NS principled refactor

The Phase 4 hybrid achieves its +17 with a controller that is, by the standards of the underlying PDE skeleton (Goertzel & Stay's HJB–NS tutorial), *ad hoc*: dye injection is unbounded (no source cap), the projection's Jacobi iteration is run with a 50-iteration ceiling that is often hit at residual ≈ 3, the projection sign convention is mathematically backwards, and there is no real transport step — the controller only *decays* u between batches.

A principled refactor of the controller along the Navier–Stokes / HJB–NS skeleton suggests the following changes (per Sec. 3.1 of the HJB–NS tutorial, Goertzel & Stay):

$$\partial_t u + (u\cdot\nabla)u - \nu\Delta u = -\nabla p - f, \quad \nabla\cdot u = 0 \tag{14}$$

$$\partial_t a + \nabla\cdot(au) = D\Delta a + s - \gamma a. \tag{15}$$

Mapped onto the bucket controller, this prescribes:

Run	Solver	Sign	Sources	h_{pressure}	Solved
Phase 4 hybrid	Jacobi	(b) $F-(p_j-p_i)$	unbounded	Leray-p	289
A-v5 (sign revert)	CG	(b)	bounded (16)	Leray-p	252
A-v6 (cap removed)	CG	(b)	uncapped	Leray-p	253
A-v7 (EMA)	CG	(b)	uncapped	EMA	246
A-v8 (Jacobi probe)	Jacobi	(b)	uncapped	EMA	245

Table 3: Sub-phase A configurations on `chainy_val_clean`. "Sign" refers to the projection update: (a) is the Helmholtz-correct $F_{\text{sol}} = F + (p_j - p_i)$; (b) is the divergence-amplifying $F_{\text{sol}} = F - (p_j - p_i)$. "EMA" is the bucket-population-rate exponential moving average, an alternative to Leray pressure for h_{pressure}. CSVs (all under `atps/logs/`): `abgc_phaseA_v5_signrevert_chainy_val_30s.csv` (252); `abgc_phaseA_v6_uncapped_chainy_val_30s.csv` (253); `abgc_phaseA_v7_realema_chainy_val_30s.csv` (246); `abgc_phaseA_v8_jacobi_chainy_val_30s.csv` (245).

1. replace Jacobi by conjugate gradient (CG) so the projection actually converges;
2. enforce the Helmholtz-correct sign $F_{\text{sol}} = F + (p_j - p_i)$;
3. add a per-batch source cap so injection is bounded;
4. add Laplacian viscosity $-\nu\Delta u$ on the net-flow field;
5. add mass-conserving upwind advection $\nabla\cdot(au)$ for the dyes;
6. add graph-Laplacian diffusion $D\Delta a$ on the dyes.

We separate these into two sub-phases. Sub-phase A keeps the controller projection-only but sweeps the four solver / signal axes (CG vs Jacobi, sign, source cap, pressure signal). Sub-phase B wires the three transport steps. Both were implemented and run end-to-end on the same 800-problem slice.

4.1 Sub-phase A — solver and signal sweeps

Five distinct configurations were evaluated on the full slice. Each configuration is one single-variable swap relative to the Phase 4 hybrid baseline.

Reading off the deltas (Table 3):

- **Sign convention** (A-v4 vs A-v5; not all rows shown): Helmholtz-correct sign $\rightarrow -10$ problems.

- **Source cap** (A-v5 vs A-v6): removing the cap of 16 adds +1.
- **Pressure signal** (A-v6 vs A-v7): swapping Leray-p for bucket-population EMA costs 7 problems.
- **Solver** (A-v7 vs A-v8): swapping CG for Jacobi (at fixed everything-else) is essentially neutral: -1 problem. The solver is not load-bearing.

A-v6 is the best Sub-phase A configuration at 253/800; even with every projection invariant honestly maintained (CG, isolated-node handling, zero-mean re-projection), the score sits 36 problems below the Phase 4 hybrid. The math is correct; the controller has lost information.

4.2 Sub-phase B — transport wired

Sub-phase B wires the three transport steps onto the A-v6 baseline. The batch update becomes

1. decay a, u;
2. build edge-list (lazily, from current support graph);
3. Laplacian viscosity on $F = u - u^\top$;
4. Hodge projection (CG);
5. mass-conserving upwind advection of a_{fwd} and a_{bwd};
6. graph-Laplacian diffusion of a_{fwd} and a_{bwd}.

Defaults: $\nu = 0.1$, $D = 0.04$, CFL safety factor 0.5.

Sub-phase B v1 — SEGV root-cause. The first wired run did not complete: 209/800 problems returned SZS status `Unknown` after fewer than 1 second of wall time, four problems hung for 5+ hours each, and the headline score was 164/800. A core dump from `coredumpctl` placed the crash inside `ClauseSetExtractEntry` in E's clause-heap machinery (`CLAUSES/ccl_clausesets.c`), not inside ABGC — which is the classic signature of a downstream consumer dereferencing a corrupted comparator. The root cause was traced to `adv_advect_dyes`'s CFL clamp: the per-edge CFL bound is correct for *one* outflow edge, but a bucket i with k outflow edges loses dye from *every* edge, and the per-edge bound does not enforce the per-node total. With ~ 10 outflow edges from a high-activity bucket, the total outflow exceeds $a_{\text{fwd}}(i)$, driving $a_{\text{fwd}}(i)$ negative. Then $h_{\text{meet}} = w_{\text{meet}} \cdot \log(\varepsilon + a_{\text{fwd}} \cdot a_{\text{bwd}})$

Run	Sign	Transport	$\|u\|_\infty$ behaviour	Solved
A-v6 (reference)	(b)	none	γ_u-bounded	253
B-v2	(b)	viscosity + advect + diffuse	overflow $\to$ inf	246
B-v3	(a) Helmholtz-correct	viscosity + advect + diffuse	bounded	238

Table 4: Sub-phase B on the same slice. Sign (a) bounds $|u|_\infty$ mathematically (the projection is divergence-removing) but loses an additional ~ 8 problems versus sign (b). Sign (b) carries the divergent signal that the H-term reads, but with viscosity + advection recycling flow into the projection, the per-batch amplification compounds and $|u|_\infty$ overflows. The score in the (b)-overflow case is still meaningful: 246 is not a random number, it is the floor of how badly the H-term can perform when the align term is saturated. CSVs: `abgc_phaseB_v2_chainy_val_30s.csv` (246); `abgc_phaseB_v3_signa_chainy_val_30s.csv` (238).

produces log(negative) = NaN; the NaN weight is inserted into E's clause heap; the heap ordering is corrupted (NaN compares falsely against every other double); `ClauseSetExtractEntry` eventually dereferences a stale pointer through the corrupted heap and SIGSEGVs.

Sub-phase B v2 — fix. Three independent guards were added.

- In `adv_advect_dyes`: replace the per-edge CFL clamp with a *per-node-total* proportional share. Each outflow edge $e = (i, j)$ transports $\phi_e = \text{cfl\_safety} \cdot (\Delta t\, u_{ij} \,/\, \sum_{e' \in \text{out}(i)} \Delta t\, u_{ie'}) \cdot a_{\text{fwd}}(i)$, so that $\sum_e \phi_e = \text{cfl\_safety} \cdot a_{\text{fwd}}(i) \le a_{\text{fwd}}(i)$ by construction.
- In `adv_diffuse_dyes`: scale the diffusion coefficient per-edge by the larger endpoint degree, so the explicit-Euler stability bound $D \cdot \max(\deg) \le 1/2$ holds even on high-degree nodes.
- In `adv_compute_H`: clamp a_{fwd} and a_{bwd} to $[0, \infty)$ before $\log(\cdot)$, and finite-check the final H before returning. This is a last-line-of-defense guard against any future code path producing a non-comparable weight.

After the fix, zero `Unknown` statuses are observed on the full slice.

Sub-phase B v2 / v3 results.

4.3 Sub-phase B telemetry

Per-batch telemetry is logged to `atps/logs/abgc_telemetry/phaseB_v2.log`. Key macro statistics across the 776 PIDs that produced telemetry:

- *Velocity-field magnitude*: median `max_u_l1` reaches 9.7×10^{275}; mean is ∞. On 99.2 % of PIDs, `max_u_l1` eventually exceeds 1. Sign (b) plus clean CG convergence plus transport's flow recycling produces a true finite-time blowup at the double-precision floating ceiling.

- *Mixed-dye bridges*: median `max_bridge_overlap` = 3556, max = 88,000. In Sub-phase A configurations the same statistic was effectively zero, because a_{fwd} and a_{bwd} lived in disjoint axiom-rooted and goal-rooted bucket sets with no transport to mix them. So the transport "works": bridges do form. The advection step is doing what it claims; the meet term has a non-trivial signal to read.

- *Projection convergence*: mean projection iterations = 48.3/50, mean residual ≈ 245 at termination. CG is hitting the iteration cap on most batches, because the transport step on the previous batch produced fresh divergence that the current projection has to absorb. The under-convergence by itself is not catastrophic — E's saturation loop is robust to noisy weights — but together with the sign-(b) amplification it produces the overflow above.

- *Progress scalar*: median `final_progress_ema` = -0.21 (mostly negative); only 21.9 % of PIDs reach `progress_ema` > 0.05. The controller's own progress signal reports decline, consistent with the score loss.

5 Discussion — why accidental signals beat clean math here

5.1 The divergence-amplifying sign convention is information

In the discrete Hodge projection, the divergence-free part of a co-chain F on the bucket graph is $F^{(a)}_{\text{sol}} = F - \nabla p$. The correct sign convention under the code's representation of L and ∇ is (a), $F^{(a)}_{\text{sol}} = F + (p_j - p_i)$ on the indexed edges (Sec. 2.5). The Phase 4 hybrid uses sign (b), $F^{(b)}_{\text{sol}} = F - (p_j - p_i)$.

Under the substitution $p_{\text{code}} \to -p_{\text{true}}$ that swaps the two signs, one can see that $F^{(b)}_{\text{sol}} = F + \nabla p_{\text{true}}$, i.e., the divergent part of F is *added*, not subtracted. The divergent part of F on the bucket graph is precisely $\nabla{\cdot}u\,|_b = \sum_j F_{bj}$: the net rate at which clauses are leaving bucket b across all parent-child links. *Clause production rate by bucket* is, intuitively, exactly what a clause-selection heuristic should reward when picking the next clause from a producing bucket: "this bucket is making children fast; pick more of its parents." The sign-(b) convention writes this rate into $u_{p\to b}$ and the H-term reads it through h_{align}.

Sign-(a) removes the divergent part of F exactly. The post-projection $u_{p\to b}$ then carries only the divergence-free residual — loops and cycles on the bucket graph — which is much weaker as a clause-production signal.

5.2 Under-convergence as the second piece of accidental information

The Phase 4 hybrid runs the Jacobi solver with an iteration cap of 50 and accepts the iterate as-is even when the residual is still ≈ 3 in the L^2 norm of `div(F)`. At active-graph sizes of 200–400 buckets, Jacobi on the graph Laplacian does not converge under the iteration cap; the controller's p is therefore *partial* — close to the true potential on small-$|p|$ regions, far from it on high-divergence regions. The sign-(b) update $F^{(b)}_{\text{sol}} = F - (p_j - p_i)$ with a small-magnitude partial p leaves most of F's divergent component intact and only locally attenuates the spikes.

This is a happy accident: Jacobi as a poor solver acts as a *frequency filter* on the divergence signal, smoothing high-frequency modes but preserving the low-frequency clause-production rate that the H-term wants. When we replace Jacobi with CG (Sub-phase A) we get full convergence in $\sim$ 10 iterations to residual $\sim 10^{-6}$; the sign-(b) update then writes the *full* divergent signal, including high-frequency artifacts. The H-term reads more loudly but the signal is noisier; in the wash, we lose ~ 10 problems.

Adding transport (Sub-phase B) closes the loop: viscosity smooths u toward an equilibrium, advection redistributes dyes along u, and the projection then has to absorb the freshly-produced divergence from the redistribution. With sign (b) this becomes a per-batch amplification loop and $|u|_\infty$ blows up. With sign (a) it stays bounded but divergence-free, and the divergent signal the H-term wants is gone. Either way, the H-term reads less.

5.3 The H-term was implicitly tuned to the Phase 4 regime

The conclusion we draw is structural, not numerical: the H-term shape meet + align + pressure was tuned (by hand, on small datasets, over many evenings) against the Phase 4 hybrid's specific signal regime — under-converged Jacobi projection, sign-(b) divergence amplification, unbounded source injection. When we replace any of these with its principled counterpart the H-term reads a structurally different signal and the tuning becomes stale. A principled controller delivers a principled signal; the consumer has to be re-tuned, perhaps even re-designed, to read it. The cost of *not* re-tuning is the +17 lift.

5.4 What kind of negative result is this?

This is not a refutation of the HJB–NS framing for inference control. The transport step does what its specification says it does: mass is conserved, bridges form, projection convergence is honest. What it does *not* do is improve a heuristic whose H-term was implicitly fitted to a different signal. The principled refactor exposes a hidden brittleness in the original hybrid: the +17 depends sensitively on the specific accident of sign (b) plus under-convergence. A reviewer who treats this controller as "mzr02 with a clever Hodge projection" underestimates how much hand-tuning the H-term encodes against the projection's idiosyncrasies.

5.5 What would a real HJB–NS clause-selection look like?

We do not believe the right next step is to fiddle with the H-term to recover the +17 on top of clean transport; that would just be re-tuning to a new accident. The right next steps, in our view, are:

- *Replace the static H-term with a learned reader* of the projected u and the dye fields, with the divergence signal explicitly factored out as a separate ranking feature.
- *Move from bucket-level to per-clause Hodge projection on the derivation graph* (the actual DAG of generated clauses), so the Helmholtz decomposition operates on the object the H-term actually reads.
- *Replace the hand-tuned ϕ-basis* (h_{conj}, h_{size}, etc.) with an ENIGMA-style learned ϕ, so the consumer can adapt to whatever signal the principled transport provides.

None of these is done in this paper. The honest contribution of Act 2 is the diagnosis, not the fix.

6 Related work

AI4REASON, MaLARea, MaSh, ENIGMA, MaSh-style. The single-heuristic baseline `mzr02` is from the AI4REASON group's IJCAR 2020 paper on hand-tuned Mizar strategies (Jakubův and Urban, IJCAR 2020). MaLARea and MaSh are premise selectors and are upstream of the saturation step we operate in. ENIGMA learns clause-selection heuristics by gradient descent on a ϕ-basis; our static H-term is a hand-engineered analog. Our sibling paper *PLN as a Common Substrate for Naive Bayes and k-NN Premise Selection* (`papers/PLN-kNN-NB.tex`) addresses the premise-selection axis on the same 800-problem slice and is the most direct point of comparison for the dataset and protocol.

Watchlist heuristics. Goal-directed watchlist heuristics in E and Vampire are the closest prior art for "bucket the unprocessed-clause set and bias toward buckets that look productive." Our 14-bit bucket signature is more coarse than a typical watchlist (which is per-symbol), and our dye+flow mechanism makes the bias quantitative rather than binary.

Fluid analogies in proof search. Goertzel and Stay's HJB–NS tutorial in `literature/Goertzel_Ben/benxiv/Fluid-AI/HJB-NS-tutorial.pdf` is the direct motivation for the Sub-phase B transport; we have implemented it as faithfully as the bucket-level setting allows and reported the result honestly. The framing as such is sound; whether it pays off empirically depends on whether the consumer of the projection is built to read it.

7 Conclusion and open work

The static-H Phase 4 hybrid is a usable clause-selection heuristic: +17 over `mzr02` on the chainy MPTP slice, +6.2 % relative, reproducible from the CSV artifacts on disk. The HJB–NS principled refactor does not exceed it; we have characterised this as a signal/consumer mismatch rather than a failure of the framing. The following questions are open and we do not address them here:

- Can the H-term be rebuilt to read a divergence-free signal as productively as it reads the divergence-amplified one? (We do not think the answer is yes for the current static H-term shape; an ENIGMA-style learned H-term might.)
- Does the framing pay off at a finer granularity? Per-clause Hodge projection on the derivation DAG would put the projection at the level the H-term actually reads, which we have not implemented.
- Does the framing pay off on a different benchmark? We have only run on chainy MPTP; harder TPTP slices or Mizar Mathematical Library splits might exhibit a different signal regime.

8 Reproducibility

8.1 Heuristic string

The HCB string passed to `eprover-H` for the headline hybrid result is the following 9:3 multiplier sum (`mzr02` entries first, AdvWeight last):

```
(
 1*ConjectureTermPrefixWeight(DeferSOS,1,3,0.1,5,0,0.1,1,4),
 1*ConjectureTermPrefixWeight(DeferSOS,1,3,0.5,100,0,0.2,0.2,4),
 1*Refinedweight(PreferWatchlist,4,300,4,4,0.7),
 1*RelevanceLevelWeight2(PreferProcessed
     ,0,1,2,1,1,1,200,200,2.5,9999.9,9999.9),
 1*StaggeredWeight(DeferSOS,1),
 1*SymbolTypeweight(DeferSOS,18,7,-2,5,9999.9,2,1.5),
 2*Clauseweight(PreferWatchlist,20,9999,4),
 2*ConjectureSymbolWeight(DeferSOS,9999,20,50,-1,50,3,3,0.5),
 2*StaggeredWeight(DeferSOS,2),
 3*AdvWeight(ConstPrio,1.0,8.0,2.0,1.0,25.0,3.0,5.0,10.0)
)
```

The `AdvWeight` arguments are $(\alpha, w_{\text{conj}}, w_{\text{depth}}, w_{\text{size}}, w_{\text{goal}}, w_{\text{meet}}, w_{\text{align}}, w_{\text{pressure}}) = (1.0, 8.0, 2.0, 1.0, 25.0, 3.0, 5.0, 10.0)$.

8.2 Runner invocation

All scores reported are produced by the runner script `atps/scripts/run_baseline_prover.sh`, invoked as

```
ABGC_LOG_PATH=atps/logs/abgc_telemetry/PHASE.log
  atps/scripts/run_baseline_prover.sh \
  PHASE_chainy_val_30s \
```

```
'eprover --free-numbers --definitional-cnf=24 --split-
   aggressive
 --simul-paramod --forward-context-sr --destructive-er-
    aggressive
 --destructive-er --prefer-initial-clauses -tKBO6 -
    winvfreqrank -c1
 -Ginvfreq -F1 --delete-bad-limit=150000000
 -WSelectMaxLComplexAvoidPosPred -H"<<heuristic string above
    >>"
 --cpu-limit={TIMEOUT} {PROBLEM}' \
atps/datasets/extended_mptp5k/chainy_val_clean \
30 \
atps/logs/PHASE_chainy_val_30s.csv \
4
```

with `ulimit-v10485760` (10 GiB virtual-memory cap) imposed at the runner-wrapper level.

8.3 CSV artifacts

CSV path (under `atps/logs/`)	Solved	Total
`baseline_e_auto_sched_chainy_val_30s.csv`	334	800
`baseline_mzr02_chainy_val_30s.csv`	272	800
`abgc_mzr02_hybrid_chainy_val_30s.csv`	289	800
`abgc_phaseA_v5_signrevert_chainy_val_30s.csv`	252	800
`abgc_phaseA_v6_uncapped_chainy_val_30s.csv`	253	800
`abgc_phaseA_v7_realema_chainy_val_30s.csv`	246	800
`abgc_phaseA_v8_jacobi_chainy_val_30s.csv`	245	800
`abgc_phaseB_v2_chainy_val_30s.csv`	246	800
`abgc_phaseB_v3_signa_chainy_val_30s.csv`	238	800

Each row is verifiable as `awk-F,'NR>1&&\protect\TU\textdollar2==1\{n++\}END\{printn+0\}'<csv>`.

8.4 Source tree

- `HEURISTICS/che_abgc.c` (E prover heuristics directory) — the ABGC controller. The active `adv_batch_update` calls only the Hodge projection by default; the Sub-phase B transport functions (`adv_build_edge_list`, `adv_viscosity_step`, `adv_advect_dyes`, `adv_diffuse_dyes`) are present in the file and statically callable, but not wired in the current production state.
- `HEURISTICS/che_abgc.h` — type definitions.
- `HEURISTICS/che_heuristics.c` — the registration site `HCBStdInit` where `AdvWeightInit` is exposed to the heuristic-string parser.

References

[1] Jan Jakubův and Josef Urban. Hammering Mizar by learning clause guidance (Mizar40 / mzr02). In *IJCAR 2020*.

[2] Stephan Schulz. System description: E 1.8. In *LPAR-19*, volume 8312 of *LNCS*, pages 735–743. Springer, 2013.

[3] Ben Goertzel and Mike Stay. HJB–Navier–Stokes for inference control: a tutorial. Working note, `literature/Goertzel_Ben/benxiv/Fluid-AI/HJB-NS-tutorial.pdf`, 2025.

[4] Daniel Kühlwein and Josef Urban. MaSh: Machine learning for Sledgehammer. In *ITP 2013*.

[5] Jan Jakubův and Josef Urban. ENIGMA: Efficient learning-based inference guiding machine. In *CICM 2017*.

[6] Josef Urban. MPTP 0.2: Design, implementation, and initial experiments. *Journal of Automated Reasoning*, 37:21–43, 2006.

[7] Mettapedia Working Notes. PLN as a common substrate for Naive Bayes and k-NN premise selection. In this directory: `papers/PLN-kNN-NB.tex`.

[8] Lek-Heng Lim. Hodge Laplacians on graphs. *SIAM Review*, 62(3):685–715, 2020.